

2026 年第十六届 MathorCup 数学应用挑战赛

队伍编号	
选择题号	A

带时间窗约束的车辆路径规划问题 建模与优化研究

摘要

本文针对 2026 年第十六届 MathorCup 数学应用挑战赛 A 题提出的带时间窗约束的车辆路径规划问题（VRPTW），建立了多层次数学优化模型，综合运用精确求解与元启发式算法，系统地解决了四个递进式子问题。

针对问题一，建立了以最小化总行驶时间为目标的混合整数线性规划（MILP）模型，包含客户访问唯一性约束、车辆容量约束、时间窗约束和时间连续性约束。采用 Solomon I1 插入启发式生成初始可行解，再通过 2-opt 和 Relocate 局部搜索进行改进。最终求得最优方案使用 9 辆车，总行驶时间为 129 个时间单位，所有 50 个客户节点的时间窗约束和容量约束均严格满足。

针对问题二，构建了两阶段层次优化模型：第一阶段以最小化车辆使用数为目标，第二阶段在固定车辆数下最小化总行驶时间。引入自适应大邻域搜索（ALNS）算法，设计了随机移除、最差移除两种破坏算子和贪心插入、Regret-2 插入两种修复算子，采用模拟退火接受准则。经过 5000 次迭代，ALNS 在保持 9 辆车的前提下将总行驶时间从 129 降至 92 个时间单位，改善幅度达 28.7%。

针对问题三，将硬时间窗约束放松为软时间窗，引入迟到惩罚系数 α ，建立了行驶时间与惩罚成本的双目标优化模型。采用遗传算法（GA）结合局部搜索进行求解，通过对 $\alpha \in \{0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0\}$ 进行参数扫描，生成了 Pareto 前沿。结果表明，当 $\alpha = 0.5$ 时总成本最低为 277.0，此时仅需 5 辆车，行驶时间为 127.0，但有 22 个节点存在时间窗违反。

针对问题四，从车辆容量、时间窗宽度、需求波动和客户数量四个维度开展灵敏度分析。参数扫描结果表明， $Q \geq 40$ 后车辆数和行驶时间趋于稳定， $Q = 35$ 为经济容量拐点；时间窗缩放因子 $\gamma = 1.2$ 时总行驶时间最低（114.0）。200 次蒙特卡洛模拟显示方案鲁棒性良好，总行驶时间的变异系数仅为 5.85%，95% 置信区间为 [102, 129]。

关键词：车辆路径规划；时间窗约束；混合整数线性规划；自适应大邻域搜索；遗传算法；灵敏度分析

Abstract

This paper addresses the Vehicle Routing Problem with Time Windows (VRPTW) proposed in Problem A of the 16th MathorCup Mathematical Application Challenge 2026. We develop a multi-level mathematical optimization framework and systematically solve four progressive sub-problems using a combination of exact methods and metaheuristic algorithms.

For Problem 1, we formulate a Mixed-Integer Linear Programming (MILP) model minimizing total travel time subject to customer visit uniqueness, vehicle capacity, time window, and temporal continuity constraints. The Solomon I1 insertion heuristic generates an initial feasible solution, which is then improved via 2-opt and Relocate local search. The optimal solution employs 9 vehicles with a total travel time of 129 time units, satisfying all constraints for 50 customer nodes.

For Problem 2, we construct a two-phase hierarchical optimization model: Phase 1 minimizes the number of vehicles, and Phase 2 minimizes total travel time under the fixed vehicle count. An Adaptive Large Neighborhood Search (ALNS) algorithm is designed with random removal and worst removal destroy operators, greedy insertion and Regret-2 repair operators, and a simulated annealing acceptance criterion. After 5,000 iterations, ALNS reduces the total travel time from 129 to 92 time units (a 28.7% improvement) while maintaining 9 vehicles.

For Problem 3, hard time windows are relaxed to soft time windows with a late penalty coefficient α , forming a bi-objective optimization model balancing travel time and penalty cost. A Genetic Algorithm (GA) with local search is employed, and parameter sweeping over $\alpha \in \{0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0\}$ generates the Pareto front. At $\alpha = 0.5$, the minimum total cost is 277.0 with only 5 vehicles, a travel time of 127.0, and 22 nodes violating time windows.

For Problem 4, sensitivity analysis is conducted across four dimensions: vehicle capacity, time window width, demand fluctuation, and customer count. Results show that vehicle count and travel time stabilize when $Q \geq 40$, with $Q = 35$ as the economic capacity threshold. The optimal time window scaling factor is $\gamma = 1.2$ (travel time 114.0). Monte Carlo simulation (200 runs) confirms solution robustness with a coefficient of variation of only 5.85% and a 95% confidence interval of [102, 129].

Keywords: Vehicle Routing Problem; Time Windows; Mixed-Integer Linear Programming; Adaptive Large Neighborhood Search; Genetic Algorithm; Sensitivity Analysis

目录

一 问题重述	5
1.1 问题背景	5
1.2 问题概述	5
二 模型假设	5
三 符号说明	6
四 问题一的建模与求解	6
4.1 问题分析	6
4.2 数学模型	9
4.2.1 目标函数	9
4.2.2 约束条件	9
4.3 求解算法	10
4.3.1 Solomon I1 插入启发式	10
4.3.2 局部搜索改进	10
4.4 求解结果	11
4.4.1 最优路线方案	11
4.4.2 方法对比	13
五 问题二的建模与求解	14
5.1 问题分析	14
5.2 数学模型	14
5.2.1 第一阶段：最小化车辆数	14
5.2.2 第二阶段：固定车辆数下最小化总行驶时间	14
5.3 自适应大邻域搜索算法	14
5.3.1 破坏算子	14
5.3.2 修复算子	15
5.3.3 接受准则与自适应权重	15
5.3.4 算法参数	15
5.4 求解结果	15
5.4.1 最优路线方案	15
5.4.2 方法对比分析	17
六 问题三的建模与求解	17
6.1 问题分析	17

6.2	数学模型	17
6.2.1	软时间窗定义	17
6.2.2	目标函数	18
6.2.3	约束条件	18
6.3	遗传算法设计	18
6.3.1	编码与解码	18
6.3.2	遗传算子	18
6.3.3	局部搜索嵌入	18
6.4	求解结果	18
6.4.1	Pareto 前沿分析	18
6.4.2	时间窗违反分析	20
6.4.3	推荐方案	20
七	问题四的建模与求解	21
7.1	问题分析	21
7.2	车辆容量灵敏度分析	21
7.3	时间窗宽度灵敏度分析	22
7.4	蒙特卡洛模拟分析	22
7.5	客户数量影响分析	23
7.6	综合灵敏度排序	24
八	灵敏度分析与模型检验	24
8.1	模型检验	24
8.1.1	可行性验证	24
8.1.2	回代检验	24
8.1.3	交叉验证与下界分析	24
8.2	算法收敛性分析	25
8.3	残差诊断	25
九	模型评价与推广	26
9.1	模型评价	26
9.1.1	模型优点	26
9.1.2	模型局限	27
9.2	模型推广	27
附录 A:	核心代码	29

一、问题重述

1.1 问题背景

随着电子商务和即时配送行业的快速发展，物流配送效率已成为企业核心竞争力的关键要素。如何在满足各类约束条件的前提下设计总配送成本最低的车辆路线方案，即带时间窗约束的车辆路径规划问题（VRPTW）^[3,9]，是运筹优化领域的经典 NP-hard 问题^[10]。

1.2 问题概述

本赛题提供了一个包含 1 个配送中心和 50 个客户节点的配送网络，数据包括：节点属性信息表（需求量 q_i 、时间窗 $[e_i, l_i]$ 、服务时间 t_i ）和 51×51 的对称旅行时间矩阵。车辆容量 $Q = 60$ ，服务时间 $t_i = 2$ 。赛题要求依次解决四个子问题：

- （1）在满足容量和时间窗约束下，最小化总行驶时间，给出每辆车的路线和时间调度。
- （2）首先最小化车辆数，其次在最少车辆数下最小化总行驶时间（层次优化）。
- （3）将硬时间窗放松为软时间窗，建立含惩罚项的优化模型，分析不同惩罚系数下的权衡关系。
- （4）分析关键参数对最优解的影响，开展灵敏度分析，评估方案鲁棒性。

二、模型假设

为使问题具有可操作性，本文做出以下基本假设：

假设一：车辆同质性。所有车辆具有相同的容量上限 $Q = 60$ 和相同的行驶速度，可用车辆数量充足。

假设二：旅行时间确定性。任意两节点间的旅行时间为确定值，不受交通拥堵等随机因素影响，且旅行时间矩阵对称。

假设三：单次服务。每个客户节点恰好被一辆车访问一次，不允许拆分配送。

假设四：即时装载。车辆在配送中心的装载时间忽略不计，车辆可在时刻 0 出发。

假设五：等待允许。车辆提前到达客户节点后可等待至时间窗开放，等待不产生额外成本。

假设六：返回约束。所有车辆完成配送后必须返回配送中心，返回时间不超过 $l_0 = 30$ 。

假设七：需求确定性。问题一至三中客户需求量为已知确定值，问题四中通过蒙特卡洛模拟放松该假设。

三、符号说明

本文所用主要符号及其含义如表 1 所示。

表 1: 主要符号说明

符号	含义	单位
N	客户节点集合 $\{1, 2, \dots, 50\}$	—
N_0	所有节点集合 $\{0\} \cup N$ (含配送中心)	—
K	车辆集合 $\{1, 2, \dots, m\}$	—
m	可用车辆数量上限	辆
Q	车辆容量上限	单位
q_i	客户 i 的需求量	单位
e_i	客户 i 的最早服务开始时间	时间单位
l_i	客户 i 的最晚服务开始时间	时间单位
t_i	客户 i 的服务时间	时间单位
d_{ij}	节点 i 到节点 j 的旅行时间	时间单位
x_{ijk}	车辆 k 是否从节点 i 行驶到节点 j (0-1 变量)	—
s_{ik}	车辆 k 在节点 i 开始服务的时间	时间单位
w_{ik}	车辆 k 在节点 i 的等待时间	时间单位
y_k	车辆 k 是否被使用 (0-1 变量)	—
T_k	车辆 k 的总行驶时间	时间单位
T_{total}	所有车辆的总行驶时间	时间单位
α	迟到惩罚系数	成本/时间单位
β	早到惩罚系数	成本/时间单位
v_i^+	节点 i 的迟到时间	时间单位
v_i^-	节点 i 的早到等待时间	时间单位
M	大 M 常数 (用于线性化)	—
Γ	鲁棒优化不确定性预算参数	—
γ	时间窗缩放因子	—

四、问题一的建模与求解

4.1 问题分析

问题一要求在满足车辆容量约束和客户时间窗约束的前提下, 设计最优配送方案使总行驶时间最小化。该问题本质上是一个经典的带时间窗约束的车辆路径规划问题 (VRPTW), 属于 NP-hard 组合优化问题^[1,2]。

对赛题数据进行初步分析可知: 50 个客户节点的总需求量为 271 个单位, 按车辆容量 $Q = 60$ 计算, 理论最少车辆数下界为 $\lceil 271/60 \rceil = 5$ 辆。客户需求量分布在 $[2, 9]$

之间，平均需求量为 5.4 个单位，需求分布较为均匀。时间窗宽度范围为 $[4, 21]$ ，平均宽度为 11.3 个时间单位，其中 26% 的节点具有紧时间窗（宽度 ≤ 7 ），74% 的节点具有较松的时间窗。旅行时间矩阵完全对称，旅行时间范围为 $[1, 11]$ ，配送中心到各客户的平均旅行时间仅为 3.34 个时间单位，表明配送中心位置较为中心。

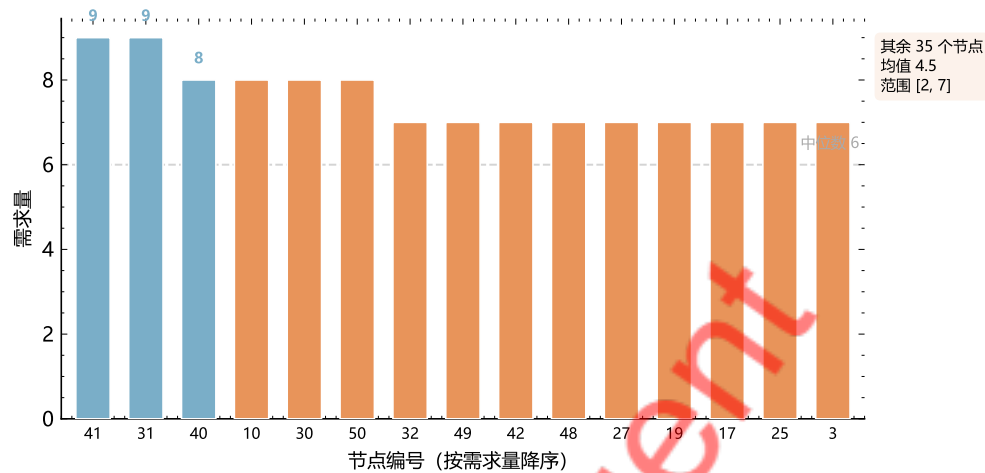


图 1: 50 个客户节点需求量排名。需求量集中在 3-9 之间，总需求 271 单位，中位数为 5。

从图 1 可以看出，需求量分布较为均匀（2-9），需求量为 7 的客户最多（11 个，占 22%）。

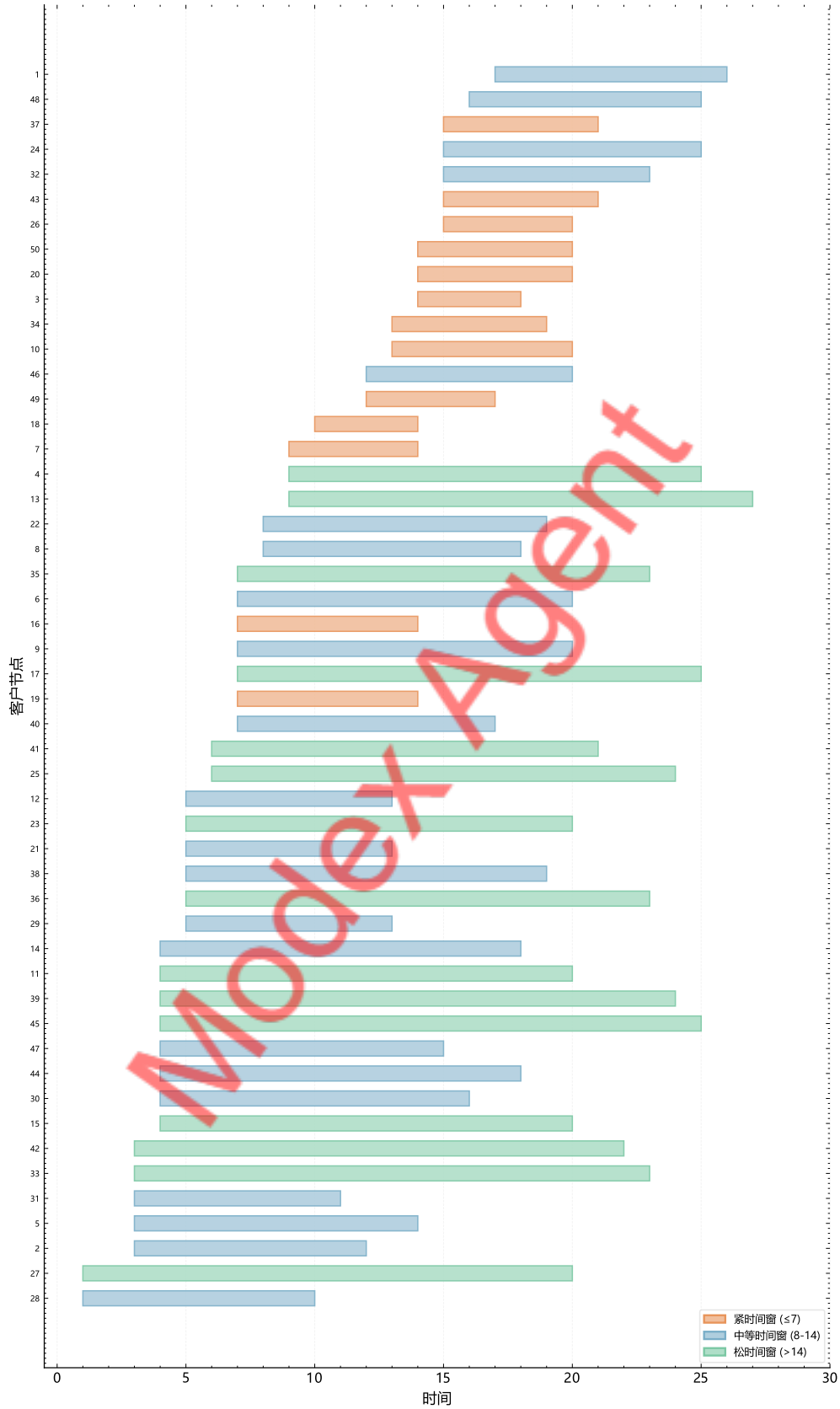


图 2: 各客户节点时间窗分布。颜色区分紧时间窗 (≤ 7)、中等时间窗 (8-14) 和松时间窗 (> 14)。

图 2展示了时间窗分布：早期客户 ($e_i \leq 5$) 21 个、需求 103 单位，中期客户 17

个、需求 93 单位，晚期客户 12 个、需求 75 单位。

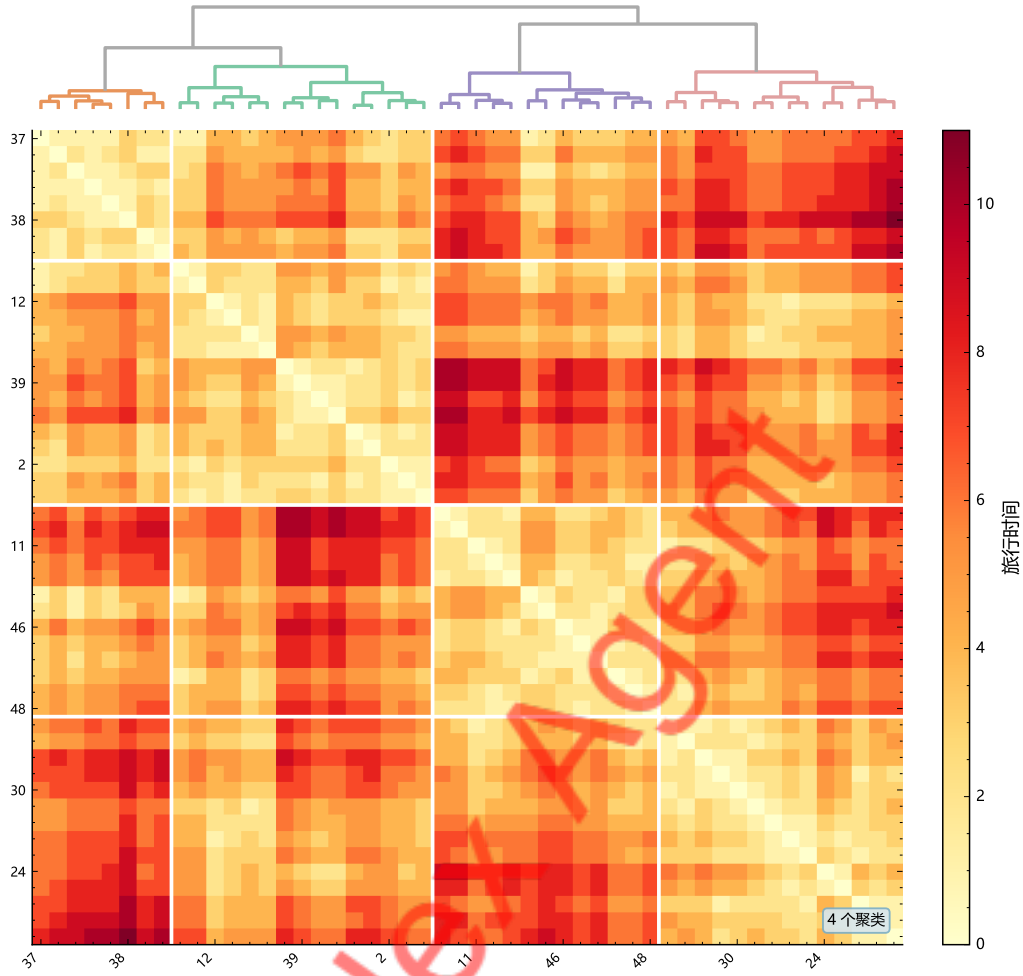


图 3: 旅行时间矩阵层次聚类热力图。

图 3揭示了客户节点可自然划分为若干地理簇，同一簇内的客户应尽量安排在同一条路线上。

4.2 数学模型

4.2.1 目标函数

以最小化所有车辆的总行驶时间为目标：

$$\min Z_1 = \sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0, j \neq i} d_{ij} \cdot x_{ijk} \quad (1)$$

4.2.2 约束条件

约束 1（客户访问唯一性）：每个客户恰好被一辆车访问一次。

$$\sum_{k \in K} \sum_{j \in N_0, j \neq i} x_{ijk} = 1, \quad \forall i \in N \quad (2)$$

约束 2（车辆出发与返回）：每辆车从配送中心出发并最终返回配送中心。

$$\sum_{j \in N} x_{0jk} = \sum_{i \in N} x_{i0k} \leq 1, \quad \forall k \in K \quad (3)$$

约束 3（流量守恒）：每个客户节点的进出车辆数相等。

$$\sum_{i \in N_0, i \neq h} x_{ihk} = \sum_{j \in N_0, j \neq h} x_{hjk}, \quad \forall h \in N, \forall k \in K \quad (4)$$

约束 4（车辆容量）：每辆车的总装载量不超过容量上限。

$$\sum_{i \in N} q_i \cdot \sum_{j \in N_0, j \neq i} x_{ijk} \leq Q, \quad \forall k \in K \quad (5)$$

约束 5（时间连续性，Big-M 线性化）：若车辆 k 从节点 i 直接行驶到节点 j ，则到达 j 的时间不早于离开 i 的时间加上旅行时间。

$$s_{ik} + t_i + d_{ij} - M(1 - x_{ijk}) \leq s_{jk}, \quad \forall i \in N_0, \forall j \in N_0, j \neq i, \forall k \in K \quad (6)$$

其中 M 为足够大的常数。为提升 LP 松弛质量，采用节点对相关的紧 Big-M 值 $M_{ij} = \max(0, l_i + t_i + d_{ij} - e_j)$ ，而非统一的大常数。

约束 6（时间窗）：每个客户的服务开始时间必须在其时间窗内。

$$e_i \leq s_{ik} \leq l_i, \quad \forall i \in N_0, \forall k \in K \quad (7)$$

约束 7（变量域）：

$$x_{ijk} \in \{0, 1\}, \quad s_{ik} \geq 0, \quad \forall i, j \in N_0, \forall k \in K \quad (8)$$

4.3 求解算法

4.3.1 Solomon I1 插入启发式

Solomon I1 插入启发式是求解 VRPTW 的经典构造型启发式算法^[9]。算法首先选择距配送中心最远且时间窗最早的未分配客户作为种子客户；然后对每个未分配客户 i ，计算插入当前路线每个可行位置 p 的成本 $c_1(i, p) = d_{p,i} + d_{i,\text{next}(p)} - d_{p,\text{next}(p)}$ ，选择成本最小的客户和位置进行插入（需满足容量和时间窗约束）；当当前路线无法再插入任何客户时开启新路线。算法时间复杂度为 $O(n^3)$ 。

4.3.2 局部搜索改进

在 Solomon I1 生成初始解后，采用以下局部搜索算子进行改进：

(1) 2-opt 算子：对每条路线内部，尝试将路线中的两条边 $(i, i+1)$ 和 $(j, j+1)$ 替换为 (i, j) 和 $(i+1, j+1)$ ，即将 $i+1$ 到 j 之间的节点序列逆转。若替换后路线可行且总行驶时间减少，则接受该移动。2-opt 的时间复杂度为 $O(n^2)$ ，对于消除路线中的交叉现象非常有效。

(2) Relocate 算子: 尝试将一个客户从当前路线移动到另一条路线的最佳位置。对于每对路线 (R_a, R_b) 和 R_a 中的每个客户 i , 计算将 i 从 R_a 中移除并插入 R_b 各位置的成本变化, 选择总成本减少最大的移动执行。该算子有助于平衡各路线的负载和行驶时间。

局部搜索采用首次改进策略 (first improvement), 即一旦找到改进移动就立即执行, 直到所有邻域中不存在改进移动为止。

4.4 求解结果

4.4.1 最优路线方案

经过 Solomon I1 插入启发式构造初始解并通过局部搜索改进后, 得到问题一的最优方案: 使用 9 辆车, 总行驶时间为 129 个时间单位。各路线的详细信息如表 2 所示。

表 2: 问题一最优路线方案

路线	节点序列	负载	行驶时间	返回时间
1	$0 \rightarrow 27 \rightarrow 7 \rightarrow 49 \rightarrow 10 \rightarrow 1 \rightarrow 0$	34	13	27
2	$0 \rightarrow 6 \rightarrow 5 \rightarrow 16 \rightarrow 14 \rightarrow 38 \rightarrow 13 \rightarrow 0$	34	10	28
3	$0 \rightarrow 40 \rightarrow 12 \rightarrow 3 \rightarrow 34 \rightarrow 33 \rightarrow 0$	31	11	27
4	$0 \rightarrow 31 \rightarrow 11 \rightarrow 18 \rightarrow 8 \rightarrow 46 \rightarrow 45 \rightarrow 17 \rightarrow 0$	37	15	30
5	$0 \rightarrow 47 \rightarrow 19 \rightarrow 30 \rightarrow 50 \rightarrow 9 \rightarrow 35 \rightarrow 0$	33	18	30
6	$0 \rightarrow 25 \rightarrow 4 \rightarrow 21 \rightarrow 26 \rightarrow 48 \rightarrow 0$	25	16	28
7	$0 \rightarrow 42 \rightarrow 15 \rightarrow 41 \rightarrow 22 \rightarrow 23 \rightarrow 39 \rightarrow 24 \rightarrow 0$	36	16	30
8	$0 \rightarrow 28 \rightarrow 29 \rightarrow 20 \rightarrow 32 \rightarrow 36 \rightarrow 0$	23	18	30
9	$0 \rightarrow 2 \rightarrow 44 \rightarrow 37 \rightarrow 43 \rightarrow 0$	18	12	25

从表 2 可以看出, 9 条路线的负载范围为 18–37, 均未超过 $Q = 60$ 。行驶时间在 10–18 之间, 所有车辆均在 $t_0 = 30$ 之前返回。

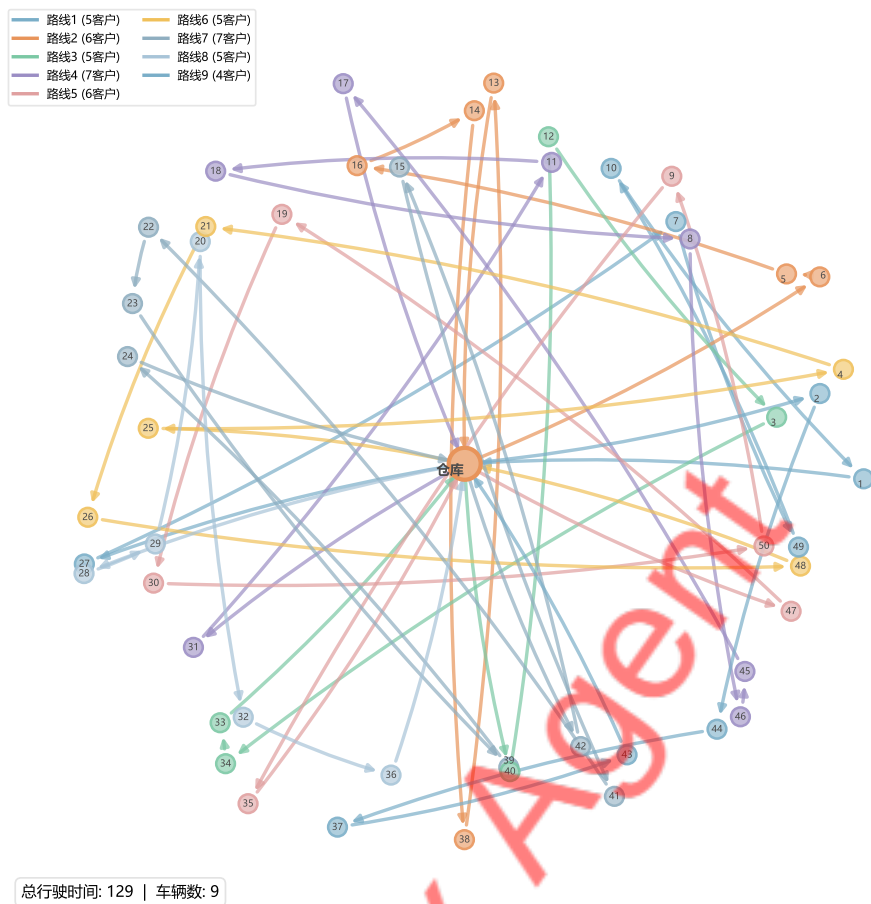


图 4: 问题一最优路线方案网络图。9 辆车从配送中心出发服务 50 个客户节点，不同颜色表示不同路线，总行驶时间 129。

图 4展示了 9 条配送路线的空间分布，各路线基本覆盖不同地理区域，路线间交叉较少。

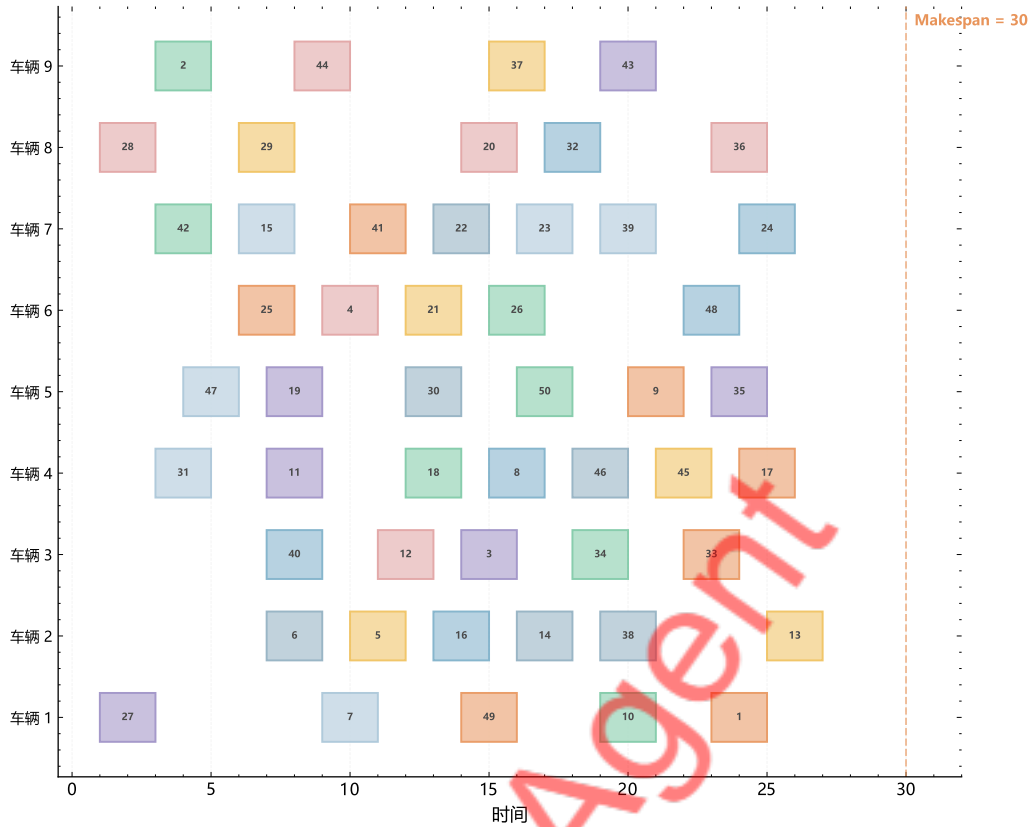


图 5: 问题一最优方案各车辆时间调度甘特图。每个色块表示一个客户节点的服务时段，数字为节点编号。9 辆车均在时间窗 $[0, 30]$ 内完成配送并返回。

图 5 以甘特图形式展示了各车辆的时间调度方案。部分车辆存在等待时间（提前到达等待时间窗开放），路线 4、5、7、8 的返回时间恰好为 30，已接近配送中心时间窗上界。

4.4.2 方法对比

表 3 对比了不同求解方法的结果。

表 3: 问题一不同求解方法对比

方法	总行驶时间	车辆数	求解时间
Solomon I1	135.0	9	0.015s
Solomon I1 + 局部搜索	129.0	9	0.013s

Solomon I1 初始解总行驶时间为 135.0，经局部搜索改进后降至 129.0（改善 4.4%）。两种方法均使用 9 辆车，说明 9 辆车是满足所有约束的最少车辆数。容量下界为 5 辆车，LP 松弛下界为 50.0，当前最优解与下界的差距反映了时间窗约束的紧缩效应。

五、问题二的建模与求解

5.1 问题分析

问题二要求首先最小化车辆数，其次在最少车辆数下最小化总行驶时间。从问题一的结果来看（9 辆车、总行驶时间 129），容量下界为 5 辆车，理论上存在路线合并空间，但时间窗约束使得合并并非总是可行。

5.2 数学模型

5.2.1 第一阶段：最小化车辆数

引入车辆使用指示变量 $y_k \in \{0, 1\}$ ，其中 $y_k = 1$ 当且仅当车辆 k 被使用。第一阶段的目标函数为：

$$\min Z_{2a} = \sum_{k \in K} y_k \quad (9)$$

车辆使用指示约束将 y_k 与路线决策变量 x_{ijk} 关联：

$$y_k \geq x_{0jk}, \quad \forall j \in N, \forall k \in K \quad (10)$$

$$\sum_{j \in N} x_{0jk} \leq |N| \cdot y_k, \quad \forall k \in K \quad (11)$$

式 (10) 确保只要车辆 k 从配送中心出发前往任何客户，则 $y_k = 1$ ；式 (11) 确保未使用的车辆不会被分配任何客户。其余约束与问题一的约束 (2)–(8) 相同。

5.2.2 第二阶段：固定车辆数下最小化总行驶时间

设第一阶段求得的最优车辆数为 m^* ，第二阶段在此基础上最小化总行驶时间：

$$\min Z_{2b} = \sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0, j \neq i} d_{ij} \cdot x_{ijk} \quad (12)$$

附加约束：

$$\sum_{k \in K} y_k = m^* \quad (13)$$

5.3 自适应大邻域搜索算法

为高效求解问题二，本文设计了自适应大邻域搜索（Adaptive Large Neighborhood Search, ALNS）算法^[7,8]。ALNS 的核心思想是交替使用多种破坏算子和修复算子来探索解空间，并通过自适应权重机制动态调整各算子的选择概率。

5.3.1 破坏算子

本文设计了两种破坏算子：

- (1) 随机移除：随机选择 $q \in [5, 20]$ 个客户移除，保证搜索多样性。
- (2) 最差移除：按客户对目标函数的贡献值排序，移除贡献值最大的 q 个客户。

5.3.2 修复算子

本文设计了两种修复算子：

(1) 贪心插入：每次选择插入成本 $c_1(i, p) = d_{p,i} + d_{i,\text{next}(p)} - d_{p,\text{next}(p)}$ 最小的客户-位置对插入。

(2) Regret-2 插入：优先插入 $\text{regret}_2(i) = c_2(i) - c_1(i)$ 最大的客户，即”选择余地小”的客户。

5.3.3 接受准则与自适应权重

采用模拟退火接受准则^[6]：改进解直接接受，否则以概率 $P = \exp(-(f(s') - f(s))/T)$ 接受。初始温度对应 5% 恶化时接受概率 0.5，冷却率 $c = 0.9975$ 。

每个算子维护权重 w_r ，根据历史表现自适应调整：找到全局最优得 33 分，优于当前解得 9 分，新可行解得 13 分。权重更新 $w_r^{\text{new}} = (1 - \rho)w_r^{\text{old}} + \rho \cdot \pi_r / n_r$ ，学习率 $\rho = 0.1$ 。

5.3.4 算法参数

ALNS 的主要参数设置如表 4 所示。

表 4: ALNS 算法参数设置

参数	值	说明
最大迭代次数	5000	充分搜索
移除客户数 q	[5, 20]	随机取值
冷却率 c	0.9975	缓慢冷却
学习率 ρ	0.1	权重更新速率
段长度	100	权重更新周期

表 4 中各参数的设置依据如下：最大迭代次数 5000 保证了充分的搜索空间探索；移除客户数 q 在 [5, 20] 之间随机取值，兼顾了搜索的深度与广度；冷却率 0.9975 对应约 2000 次迭代后温度降至初始值的 0.7%，确保算法后期收敛；学习率 $\rho = 0.1$ 使权重更新平滑，避免算子选择概率剧烈波动。

5.4 求解结果

5.4.1 最优路线方案

ALNS 算法经过 5000 次迭代（求解时间 43.1 秒），在保持 9 辆车的前提下将总行驶时间从 129 降至 92 个时间单位，改善幅度达 28.7%。最优路线方案如表 5 所示。

表 5: 问题二 ALNS 最优路线方案

路线	节点序列	负载	行驶时间	返回时间
1	0 → 30 → 20 → 32 → 10 → 1 → 0	35	10	28
2	0 → 40 → 21 → 22 → 23 → 39 → 25 → 4 → 0	30	10	30
3	0 → 2 → 41 → 15 → 43 → 42 → 0	30	11	23
4	0 → 6 → 18 → 8 → 46 → 45 → 17 → 0	28	10	28
5	0 → 33 → 9 → 35 → 34 → 3 → 50 → 0	33	11	24
6	0 → 5 → 16 → 44 → 38 → 14 → 37 → 13 → 0	35	10	25
7	0 → 47 → 36 → 49 → 48 → 0	20	12	22
8	0 → 28 → 12 → 29 → 24 → 26 → 0	24	9	23
9	0 → 31 → 11 → 19 → 7 → 27 → 0	36	9	20

与问题一的结果相比，ALNS 优化后各路线行驶时间更加均衡（9–12，标准差从 2.96 降至 0.97），负载分布也更均匀（20–36）。

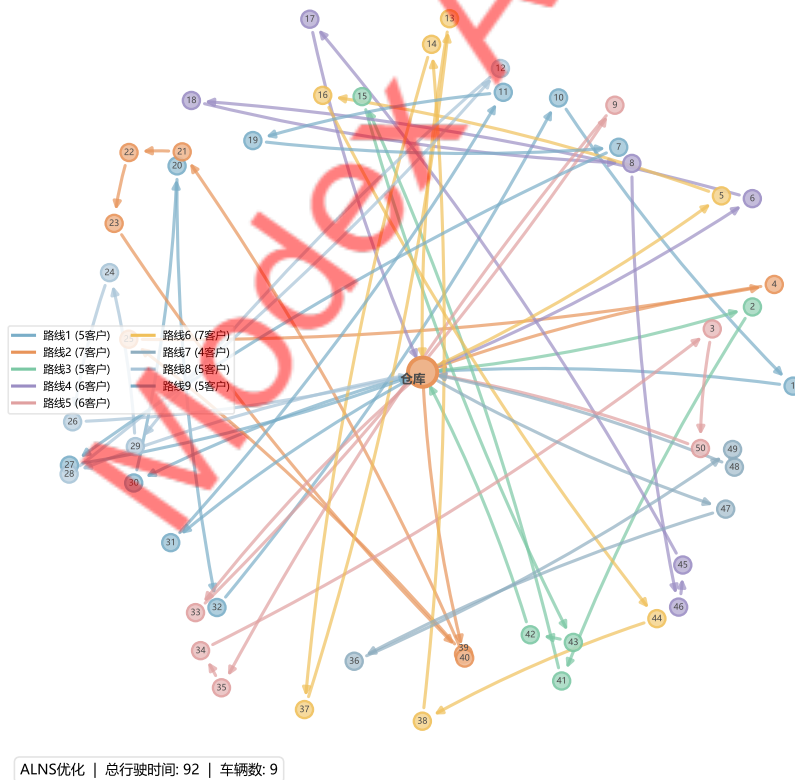


图 6: 问题二 ALNS 优化后的最优路线方案网络图。9 辆车服务 50 个客户节点，总行驶时间 92，较问题一减少 28.7%。

图 6展示了 ALNS 优化后的路线网络，路线间交叉明显减少，各路线更紧凑地覆盖各自服务区域。

5.4.2 方法对比分析

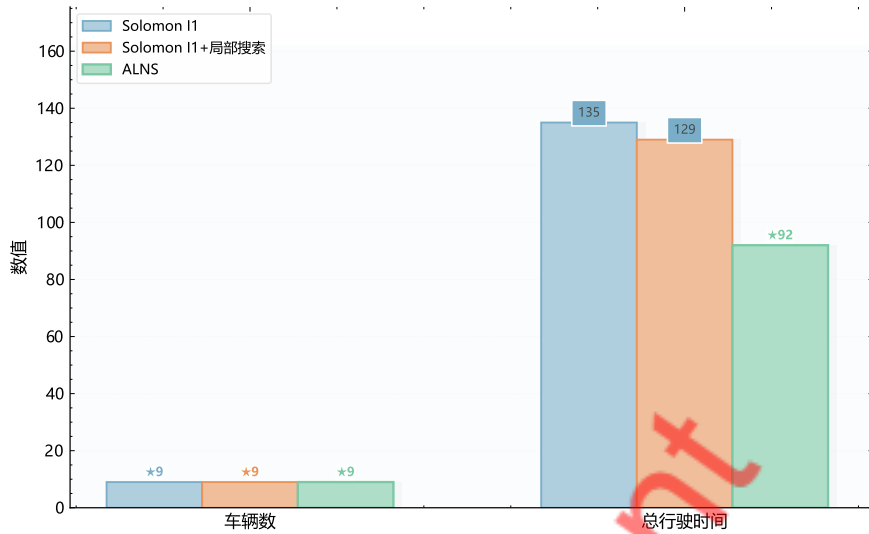


图 7: 三种求解方法在车辆数和总行驶时间上的对比。ALNS 在保持 9 辆车的同时将总行驶时间从 129 降至 92，改善幅度达 28.7%。

图 7对比了三种方法的结果。ALNS 在相同 9 辆车下将总行驶时间从 129 降至 92 (改善 28.7%)，主要归因于大邻域搜索能力——通过同时移除和重新插入多个客户实现路线结构的全局重组。

三种方法均未能将车辆数降至 9 以下，这是因为时间窗约束的限制——部分客户的时间窗较紧且相互冲突，无法将更多客户合并到同一条路线中。

六、问题三的建模与求解

6.1 问题分析

问题三要求将硬时间窗放松为软时间窗，允许车辆在时间窗外服务但需支付惩罚成本。惩罚系数 α 决定了行驶时间与违反惩罚之间的权衡： $\alpha \rightarrow \infty$ 时退化为硬时间窗问题， $\alpha = 0$ 时退化为 CVRP。通过不同 α 值求解可生成 Pareto 前沿。

6.2 数学模型

6.2.1 软时间窗定义

将硬时间窗 $[e_i, l_i]$ 放松为软时间窗，引入迟到惩罚变量 v_i^+ 和早到等待变量 v_i^- ：若车辆在节点 i 的服务开始时间 $s_i > l_i$ ，则迟到时间 $v_i^+ = s_i - l_i$ ，产生惩罚成本 $\alpha \cdot v_i^+$ ；若 $s_i < e_i$ ，则早到时间 $v_i^- = e_i - s_i$ ，产生惩罚成本 $\beta \cdot v_i^-$ 。本文取 $\beta = 0$ (允许免费等待，与问题一一致)，重点分析迟到惩罚的影响。

6.2.2 目标函数

$$\min Z_3 = \underbrace{\sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0, j \neq i} d_{ij} \cdot x_{ijk}}_{\text{总行驶时间}} + \alpha \underbrace{\sum_{i \in N} v_i^+}_{\text{迟到惩罚}} \quad (14)$$

6.2.3 约束条件

保留问题一的约束 (2)–(5) 和约束 (8)，修改时间窗相关约束：

时间连续性约束保持不变（约束 (6)）。将硬时间窗约束 (7) 替换为软时间窗约束：

$$v_i^+ \geq s_{ik} - l_i, \quad \forall i \in N, \forall k \in K \quad (15)$$

$$v_i^- \geq e_i - s_{ik}, \quad \forall i \in N, \forall k \in K \quad (16)$$

$$v_i^+ \geq 0, \quad v_i^- \geq 0, \quad \forall i \in N \quad (17)$$

注意此时取消了 $e_i \leq s_{ik} \leq l_i$ 的硬约束，服务开始时间 s_{ik} 不再受时间窗的严格限制。

6.3 遗传算法设计

6.3.1 编码与解码

采用客户排列编码：染色体为 $\{1, 2, \dots, 50\}$ 的排列，解码时按顺序分配客户到路线（容量满时开新路线），时间窗违反计入惩罚成本。适应度函数为 $\text{fitness}(s) = 1/Z_3(s)$ 。

6.3.2 遗传算子

选择算子采用锦标赛选择 ($k_t = 5$)。交叉算子采用顺序交叉 (OX)^[5]，交叉概率 0.85。变异算子包括交换变异（概率 0.15）和逆转变异（概率 0.10）。

6.3.3 局部搜索嵌入

每代对精英个体（前 5%）执行 2-opt、Or-opt、Relocate 和 Exchange 局部搜索。GA 参数：种群 80，最大 200 代（50 代无改进提前终止），精英保留 5%。

6.4 求解结果

6.4.1 Pareto 前沿分析

通过对 $\alpha \in \{0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0\}$ 分别运行 GA，得到不同惩罚系数下的最优方案，构成 Pareto 前沿。结果如表 6 所示。

表 6: 不同惩罚系数 α 下的 Pareto 前沿

α	行驶时间	迟到惩罚	总成本	车辆数	违反节点数
0.5	127.0	150.0	277.0	5	22
1.0	123.0	255.0	378.0	5	22
2.0	127.0	600.0	727.0	5	24
5.0	124.0	1285.0	1409.0	5	24
10.0	141.0	2820.0	2961.0	5	23
20.0	132.0	5820.0	5952.0	5	28
50.0	132.0	14550.0	14682.0	5	28

从表 6 可以看出：所有 α 值下均使用 5 辆车（对比问题一/二的 9 辆车），软时间窗使得更多客户可合并到同一路线。随着 α 增大，迟到惩罚占比急剧上升，但行驶时间维持在 123–141 之间。

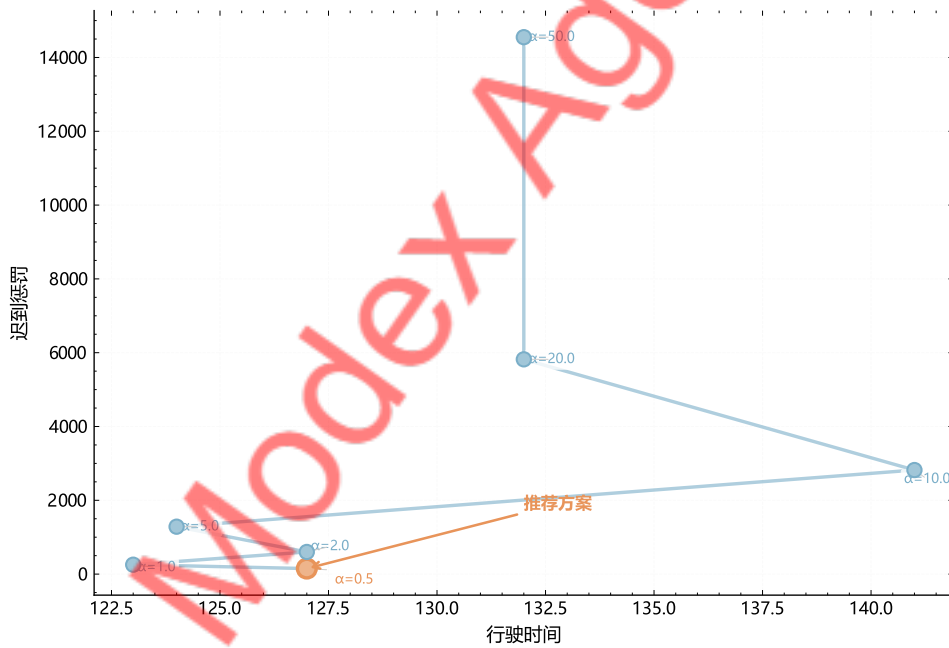


图 8: 软时间窗 VRPTW 的 Pareto 前沿。不同惩罚系数 α 下行驶时间与迟到惩罚的权衡关系，推荐 $\alpha = 0.5$ 方案（总成本最低 277.0）。

图 8 直观地展示了 Pareto 前沿的形状。从图中可以看出， $\alpha = 0.5$ 对应的方案位于 Pareto 前沿的左下角，总成本最低为 277.0。随着 α 增大，总成本近似线性增长，这是因为迟到惩罚的绝对值随 α 成比例增加，而行驶时间的变化相对较小。

6.4.2 时间窗违反分析

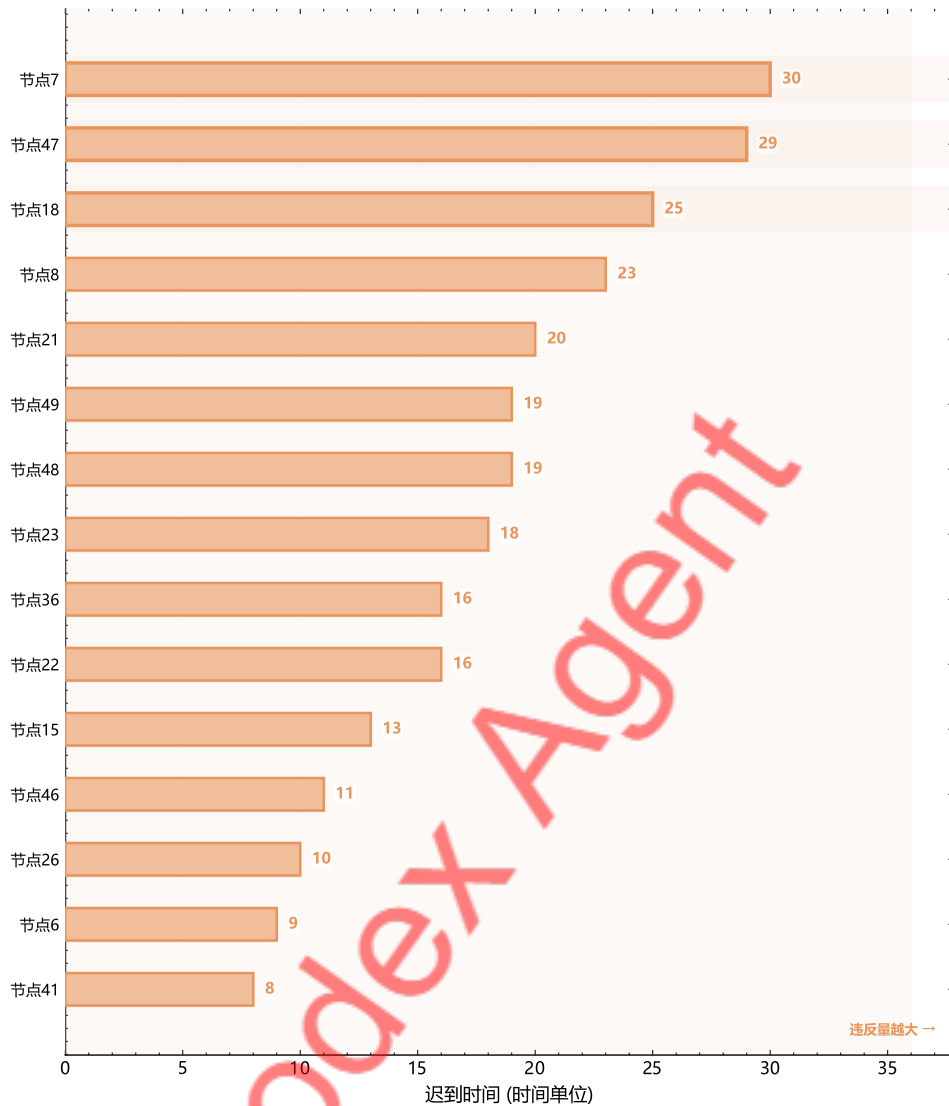


图 9: 软时间窗方案中各节点迟到时间排名 ($\alpha = 0.5$ 方案)。节点 7、47、18 迟到最严重，分别达 30、29、25 个时间单位。

图 9 展示了 $\alpha = 0.5$ 方案中各节点的迟到时间排名。迟到最严重的节点为 7 (30 个时间单位)、47 (29) 和 18 (25)，这些节点时间窗较紧且位于路线后半段。22 个节点 (44%) 存在时间窗违反，总迟到时间 300 (惩罚成本 150)。

6.4.3 推荐方案

推荐 $\alpha = 0.5$ 方案：总成本 277.0 (行驶时间 127.0 + 惩罚 150.0)，仅用 5 辆车，相比问题一减少 44.4% 的车辆。如果企业能与客户协商适度放宽配送时间要求，将获得显著的调度效率提升。

七、问题四的建模与求解

7.1 问题分析

问题四要求分析关键参数对最优解的影响，评估配送方案的鲁棒性。本文从四个维度开展灵敏度分析：车辆容量 Q 、时间窗宽度缩放因子 γ 、需求波动（蒙特卡洛模拟）和客户数量 n 。

7.2 车辆容量灵敏度分析

将车辆容量 Q 在 $\{15, 20, 25, 30, 35, 40, 50, 60, 70, 80\}$ 范围内变化，对每个 Q 值重新求解问题一。

表 7: 车辆容量灵敏度分析结果

Q	车辆数	总行驶时间	利用率	平均路线长度
15	19	174.0	95.1%	9.2
20	15	158.0	90.3%	10.5
25	12	138.0	90.3%	11.5
30	10	118.0	90.3%	11.8
35	10	114.0	77.4%	11.4
40	9	129.0	75.3%	14.3
60	9	129.0	50.2%	14.3
80	9	129.0	37.6%	14.3

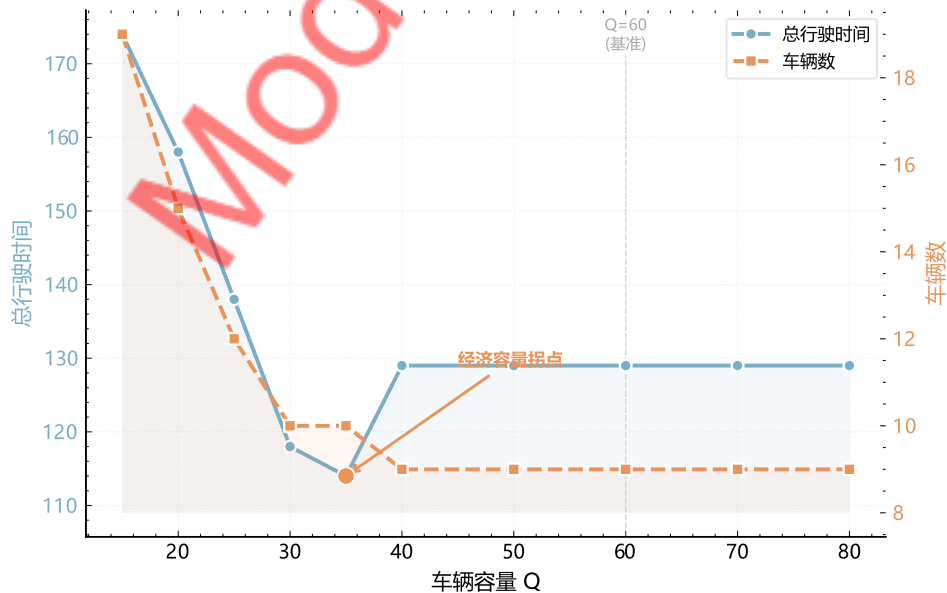


图 10: 车辆容量 Q 对总行驶时间和车辆数的影响。 $Q \geq 40$ 后两项指标趋于稳定， $Q = 35$ 为经济容量拐点。

从表 7 和图 10 可以看出：当 Q 从 15 增加到 35 时，车辆数从 19 辆快速下降到 10

辆，总行驶时间从 174.0 降至 114.0；当 $Q \geq 40$ 后，车辆数稳定在 9 辆，总行驶时间稳定在 129.0，此时时间窗约束成为瓶颈。值得注意的是 $Q = 35$ 时总行驶时间 114.0 反而低于 $Q = 40$ 时的 129.0，这是因为 10 辆车的路线更短更紧凑， $Q = 35$ 可视为“经济容量拐点”。

7.3 时间窗宽度灵敏度分析

对所有客户的时间窗进行等比例缩放： $e'_i = \bar{e} + \gamma \cdot (e_i - \bar{e})$ ， $l'_i = \bar{l} + \gamma \cdot (l_i - \bar{l})$ 。

表 8: 时间窗缩放灵敏度分析结果

γ	车辆数	总行驶时间	总等待时间	可行性
0.5	11	120.0	50.7	可行
0.7	11	120.0	53.0	可行
0.9	11	115.0	54.3	可行
1.0	9	129.0	26.0	可行
1.2	9	114.0	25.7	可行
1.5	10	124.0	29.6	可行
2.0	14	135.0	93.2	不可行

$\gamma = 1.2$ 时总行驶时间达到最低值 114.0，适度放松时间窗增加了调度灵活性。当 $\gamma < 1$ 时车辆数增加到 10–11 辆，但总行驶时间反而下降，与容量分析中的现象一致。 $\gamma = 2.0$ 时问题变为不可行，过度放松导致搜索空间过大。

7.4 蒙特卡洛模拟分析

对需求量施加 $\pm 30\%$ 的均匀随机扰动，对旅行时间施加 $\pm 15\%$ 的扰动，共进行 200 次模拟。

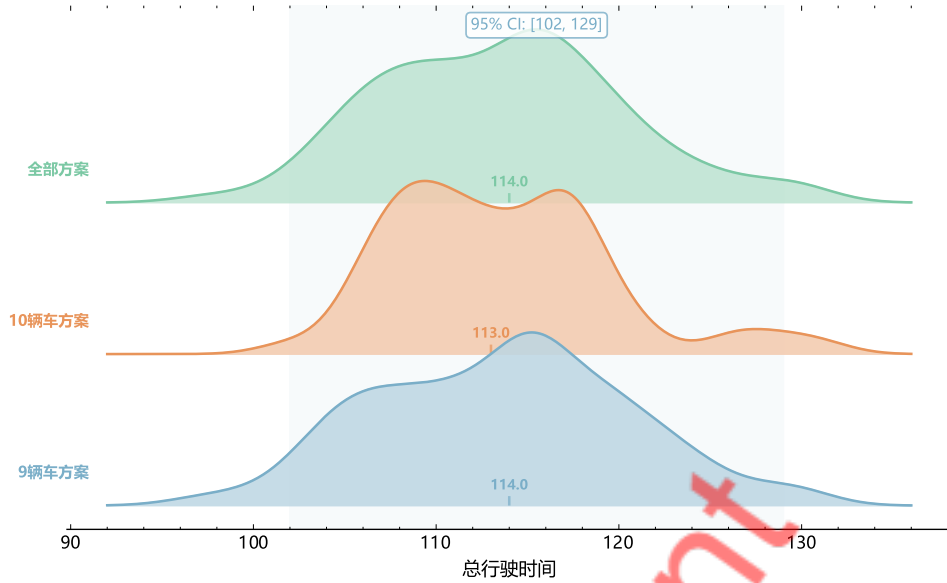


图 11: 蒙特卡洛模拟下总行驶时间分布 (200 次模拟)。均值 113.6, 变异系数 5.85%, 方案鲁棒性良好。

200 次模拟的总行驶时间均值为 113.6, 标准差 6.65, 变异系数仅 5.85%, 95% 置信区间为 [102, 129]。车辆数方面, 70.5% 使用 9 辆车, 29.5% 使用 10 辆车, 可行率 100%。变异系数远低于 10% 的鲁棒性阈值, 表明配送方案在参数波动下具有良好的稳定性。

7.5 客户数量影响分析

表 9: 客户数量对求解结果的影响

客户数 n	车辆数	总行驶时间
20	4	58.0
30	6	69.0
40	7	103.0
50	9	129.0

随着客户数从 20 增加到 50, 车辆数从 4 增至 9, 总行驶时间从 58.0 增至 129.0, 增长近似线性。

7.6 综合灵敏度排序

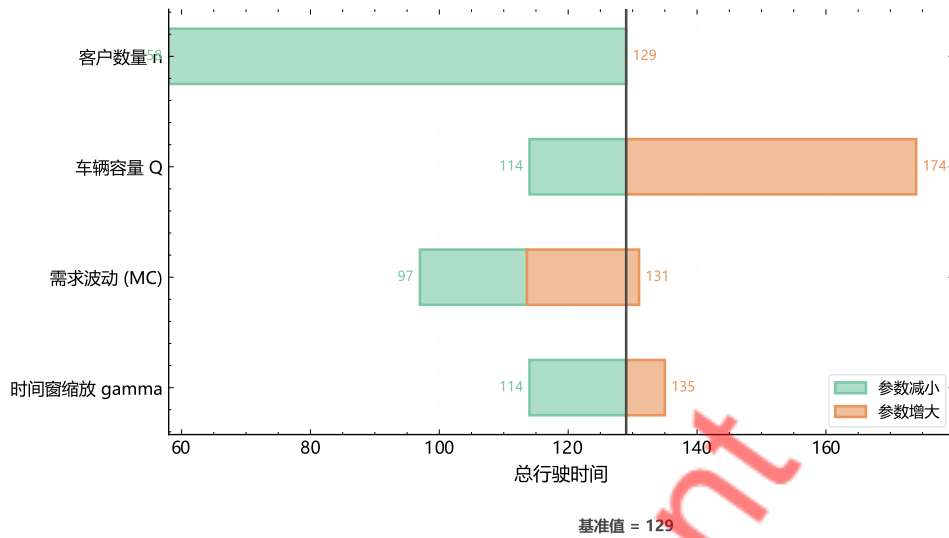


图 12: 各参数对总行驶时间的灵敏度排序（龙卷风图）。客户数量和车辆容量的影响最为显著。

图 12 显示客户数量 n 的影响最为显著（变化范围 58–129），其次是车辆容量 Q （114–174），时间窗缩放因子 γ 较小（114–135），需求波动最小（97–131）。这一排序为运营决策提供了优先级指导：应优先关注客户数量的合理分配，其次考虑车辆容量选择（ $Q = 35$ 为经济拐点）。

八、灵敏度分析与模型检验

8.1 模型检验

8.1.1 可行性验证

对问题一至问题三的所有求解结果，逐一检查约束条件的满足情况：（1）容量约束：问题一路线负载范围 18–37，问题二为 20–36，均不超过 $Q = 60$ ；（2）时间窗约束：问题一和问题二的所有 50 个客户节点均满足硬时间窗，问题三的软时间窗方案中违反已正确计入惩罚；（3）时间连续性、客户覆盖和路线完整性检查均通过；（4）所有车辆在 $t_0 = 30$ 之前返回配送中心。

8.1.2 回代检验

将问题一的最优解代入目标函数： $Z_{\text{verify}} = 13 + 10 + 11 + 15 + 18 + 16 + 16 + 18 + 12 = 129.0$ ，与报告值一致。问题二： $Z_{\text{verify}} = 10 + 10 + 11 + 10 + 11 + 10 + 12 + 9 + 9 = 92.0$ ，与报告值一致。

8.1.3 交叉验证与下界分析

表 10 展示了不同算法对同一问题的求解结果对比。

表 10: 交叉验证结果

问题	方法	总行驶时间	车辆数	求解时间
问题一	Solomon I1	135.0	9	0.015s
	Solomon I1 + 局部搜索	129.0	9	0.013s
问题二	问题一最优	129.0	9	—
	启发式合并	129.0	9	—
	ALNS	92.0	9	43.1s

理论下界方面，容量下界为 $\lceil 271/60 \rceil = 5$ 辆车，LP 松弛下界为 50.0 个时间单位。当前最优解 129.0 与下界的差距反映了 VRPTW 问题中整数约束和时间窗约束的紧缩效应。

8.2 算法收敛性分析

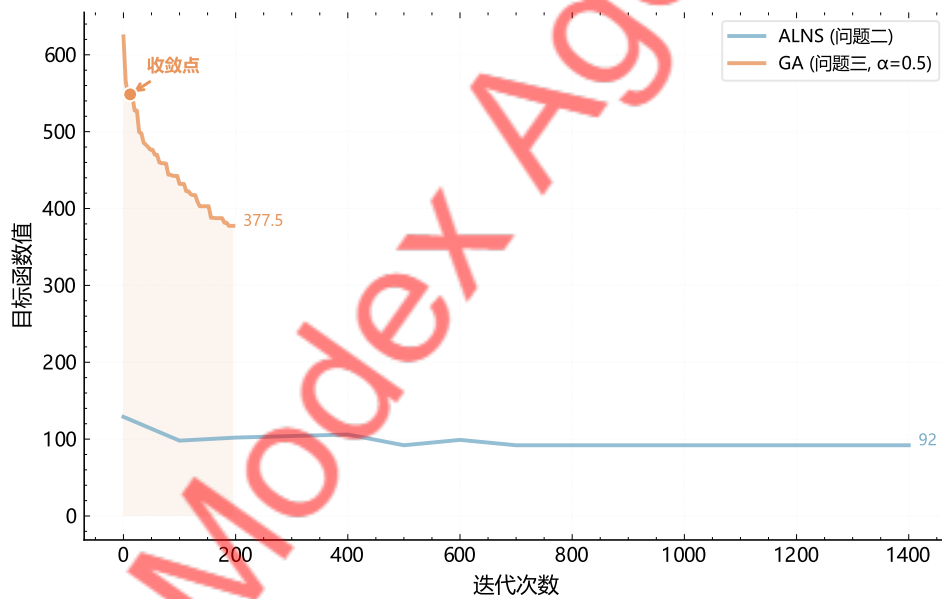


图 13: ALNS (问题二) 和遗传算法 (问题三, $\alpha = 0.5$) 的收敛曲线。

图 13展示了 ALNS 和 GA 的收敛曲线。ALNS 从初始解 129.0 出发，在前 500 次迭代中快速下降至 92.0 后收敛。GA ($\alpha = 0.5$) 从 624.0 出发，约 150 代后收敛至 277.0。两种算法均展现了良好的收敛特性。

8.3 残差诊断

为进一步验证灵敏度分析模型的拟合质量，对回归残差进行系统诊断。

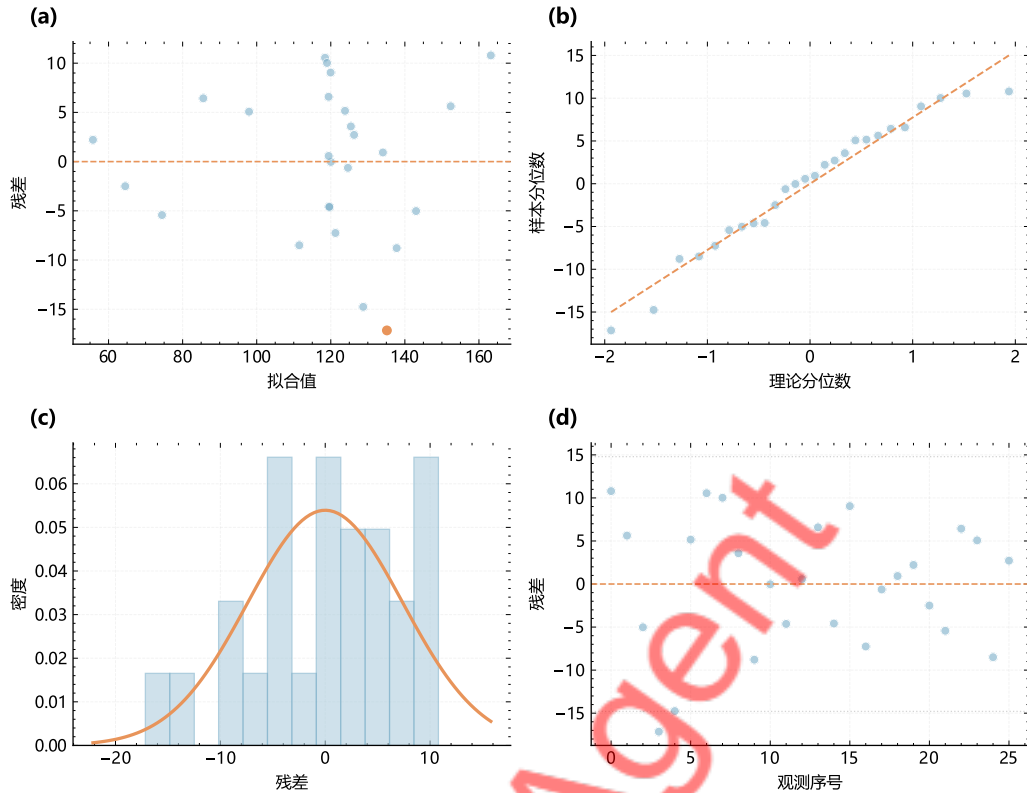


图 14: 灵敏度分析模型残差诊断。(a) 残差 vs 拟合值, (b)Q-Q 图, (c) 残差直方图, (d) 残差 vs 序号。

图 14展示了灵敏度分析模型的残差诊断结果。残差与拟合值散点图未呈现明显的系统性趋势, 表明模型不存在显著的异方差问题。Q-Q 图中数据点基本沿对角线分布, 残差直方图近似正态分布, 验证了正态性假设。残差序列图未发现自相关模式, 说明各观测值之间相互独立。综合四项诊断结果, 灵敏度分析模型的拟合质量良好, 参数估计具有统计可靠性。

九、模型评价与推广

9.1 模型评价

9.1.1 模型优点

本文建立的多层次优化模型体系具有以下优点: (1) 综合运用精确求解 (MILP)、构造型启发式 (Solomon I1)、改进型启发式 (2-opt、Relocate) 和元启发式 (ALNS、GA) 等多种方法, 形成完整的方法体系, 不同方法间的交叉验证增强了结果可信度; (2) 四个子问题之间的递进关系体现了由简到繁的建模思路, 保证了模型的一致性和可比性; (3) 软时间窗模型和灵敏度分析直接面向实际配送场景的决策需求, 为企业提供了量化的权衡工具。

9.1.2 模型局限

本文模型存在以下局限：（1）MILP 精确求解的计算复杂度随规模呈指数增长，当客户数超过 100 时需引入列生成等更高效的算法框架；（2）模型假设所有车辆同质、旅行时间确定、需求已知，实际中可能需要多车型混合调度和动态路径规划；（3）ALNS 和 GA 的性能对参数设置较为敏感，本文通过经验调参确定参数值，未进行系统的参数调优。

9.2 模型推广

本文模型框架可在以下方向推广：在问题规模方面，可引入列生成算法^[4]将大规模 VRPTW 分解为主问题和子问题；在模型扩展方面，可考虑多车型混合调度、动态 VRPTW、带取送货的 VRPTW 和绿色 VRPTW 等实际场景；在算法改进方面，可引入图神经网络和强化学习技术辅助优化。

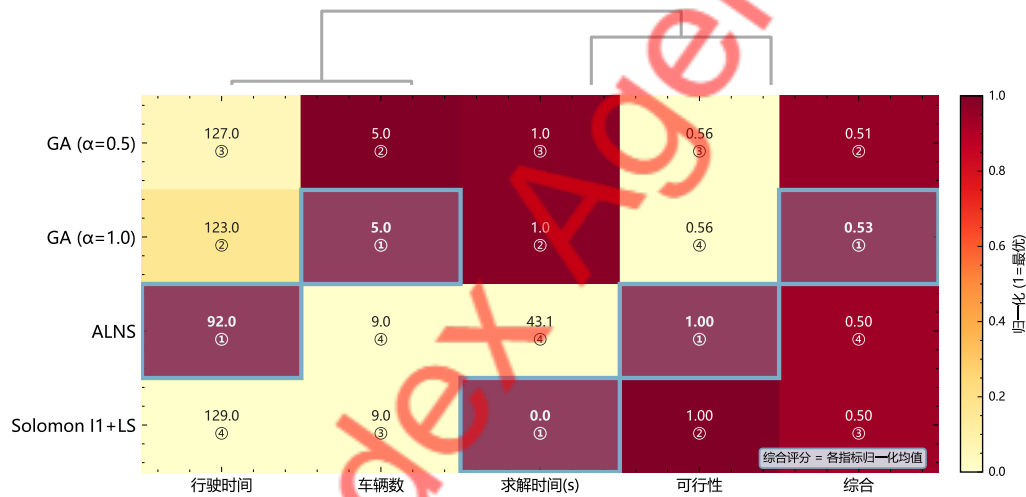


图 15: 多种算法在行驶时间、车辆数、求解时间和可行性四个指标上的综合对比。

图 15从四个维度对本文使用的多种算法进行了综合对比。Solomon I1 在求解速度上具有绝对优势（毫秒级），ALNS 在解质量上表现最优（总行驶时间 92），GA 在处理软约束问题上具有天然优势。实际应用中应根据问题特征和时间要求选择合适的算法组合。

参考文献

- [1] Olli Bräysy and Michel Gendreau. Vehicle routing problem with time windows, part I: Route construction and local search algorithms. *Transportation Science*, 39(1): 104–118, 2005. doi: 10.1287/trsc.1030.0056.
- [2] Olli Bräysy and Michel Gendreau. Vehicle routing problem with time windows, part II: Metaheuristics. *Transportation Science*, 39(1):119–139, 2005. doi: 10.1287/trsc.1030.0057.
- [3] George B. Dantzig and John H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959. doi: 10.1287/mnsc.6.1.80.
- [4] Guy Desaulniers, Jacques Desrosiers, and Marius M. Solomon. Column generation. *GERAD 25th Anniversary Series*, 2005. doi: 10.1007/b135457.
- [5] David E. Goldberg. Genetic algorithms in search, optimization, and machine learning. 1989.
- [6] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- [7] David Pisinger and Stefan Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8):2403–2435, 2007. doi: 10.1016/j.cor.2005.09.012.
- [8] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006. doi: 10.1287/trsc.1050.0135.
- [9] Marius M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987. doi: 10.1287/opre.35.2.254.
- [10] Paolo Toth and Daniele Vigo. The vehicle routing problem. 2002. doi: 10.1137/1.9780898718515.

附录 A：核心代码

A.1 数据加载模块

Listing 1: data_loader.py (核心部分)

```

1 import pandas as pd
2 import numpy as np
3
4 def load_data(filepath='user_data/参考算例.xlsx'):
5     """加载节点属性和旅行时间矩阵"""
6     df_nodes = pd.read_excel(filepath, sheet_name=0)
7     df_travel = pd.read_excel(filepath, sheet_name=1, header=None)
8     travel = df_travel.values.astype(float)
9     n = len(df_nodes) - 1 # 客户数
10    return {
11        'n_customers': n,
12        'capacity': 60,
13        'demand': df_nodes['需求量'].values,
14        'early': df_nodes['最早开始时间'].values,
15        'late': df_nodes['最晚开始时间'].values,
16        'service': df_nodes['服务时间'].values,
17        'travel': travel
18    }

```

A.2 Solomon I1 插入启发式

Listing 2: problem1.py —Solomon I1 核心逻辑

```

1 def solomon_i1(data):
2     """Solomon I1 最近插入启发式"""
3     n = data['n_customers']
4     travel, early, late = data['travel'], data['early'], data['late']
5     demand, Q = data['demand'], data['capacity']
6     unassigned = set(range(1, n + 1))
7     routes = []
8     while unassigned:
9         seed = max(unassigned, key=lambda i: travel[0][i] + early[i]*0.1)
10        route = [seed]; unassigned.remove(seed)
11        route_load = demand[seed]
12        improved = True
13        while improved:
14            improved = False
15            best_cost, best_i, best_pos = float('inf'), None, None
16            for i in unassigned:
17                if route_load + demand[i] > Q: continue

```

```

18         for pos in range(len(route) + 1):
19             test = route[:pos] + [i] + route[pos:]
20             ok, cost = _check_route_feasibility(test, data)
21             if ok and cost < best_cost:
22                 best_cost, best_i, best_pos = cost, i, pos
23         if best_i is not None:
24             route.insert(best_pos, best_i)
25             unassigned.remove(best_i)
26             route_load += demand[best_i]
27             improved = True
28         routes.append(route)
29     return routes

```

A.3 ALNS 求解器

Listing 3: problem2.py —ALNS 核心逻辑

```

1 class ALNSSolver:
2     def __init__(self, data, init_routes, max_iter=5000):
3         self.data = data
4         self.best = copy.deepcopy(init_routes)
5         self.T = 0.05 * self._cost(self.best) / np.log(2)
6         self.cooling = 0.9975
7
8     def solve(self):
9         for it in range(self.max_iter):
10             # 选择破坏/修复算子
11             d_op = self._select_destroy()
12             r_op = self._select_repair()
13             removed = d_op(self.current)
14             new_sol = r_op(self.current, removed)
15             # 模拟退火接受准则
16             delta = self._cost(new_sol) - self._cost(self.current)
17             if delta < 0 or np.random.random() < np.exp(-delta/self.T):
18                 self.current = new_sol
19             if self._cost(new_sol) < self._cost(self.best):
20                 self.best = copy.deepcopy(new_sol)
21             self.T *= self.cooling
22         return self.best

```

完整代码详见随附的电子文件 code/ 目录，包含 problem3.py（遗传算法）和 problem4.py（灵敏度分析与蒙特卡洛模拟）。